# Moneta Boot Generic Reporting Engine

***Clean Architecture | Pluggable | High Performance | CQRS-Based Design***

## 1. Overview

This document describes the architecture of a high-performance, pluggable reporting engine built using Moneta Boot (Spring Boot 3.x) following Clean Architecture principles. The system ensures that adding new reports (UDMs) does not require modifying core logic.

## 2. Architecture Layers (Clean Architecture)

- API Layer – Controllers, DTOs, Request/Response mapping
- Core Layer – Domain models, Orchestrator, Ports (Interfaces)
- Reports Layer – Pluggable UDM strategies implementing ReportProvider
- Infrastructure Layer – Database adapters, Audit adapters, Security components

## 3. Design Patterns Used

- Strategy Pattern – Each UDM implements ReportProvider
- Factory / Registry Pattern – Dynamic provider selection via Spring Map injection
- Template Method Pattern – Common execution flow in AbstractReportProvider
- Specification Pattern – Dynamic filter construction
- Adapter Pattern – Multiple data source integration
- Event-Driven Pattern – Async audit logging

## 4. Data Flow Summary

- Client sends POST /api/v1/reports/{category}
- Auth Interceptor validates request
- Report Orchestrator invokes ReportProvider via Factory/Registry

- Provider calls Repository Port
- Infrastructure Adapter executes DB query
- Results formatted and returned as paginated JSON
- Async audit event published to Audit Adapter

## 5. Performance & Scalability Strategies

- Streaming large result sets (JdbcTemplate queryForStream)
- Pagination using Pageable or offset/limit
- Caching layer (Caffeine / Redis)
- Async job execution for heavy reports
- Read replicas or data warehouse for reporting workloads
- Response compression enabled

## 6. Deployment Strategy

- Dockerized container deployment
- Private cloud hosting
- Observability via Micrometer + Prometheus
- Distributed tracing via OpenTelemetry
- Structured logging to ELK/Splunk